

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Muhammad Arifin Umasangadji - 5024241083

2026

1 Langkah-Langkah Percobaan

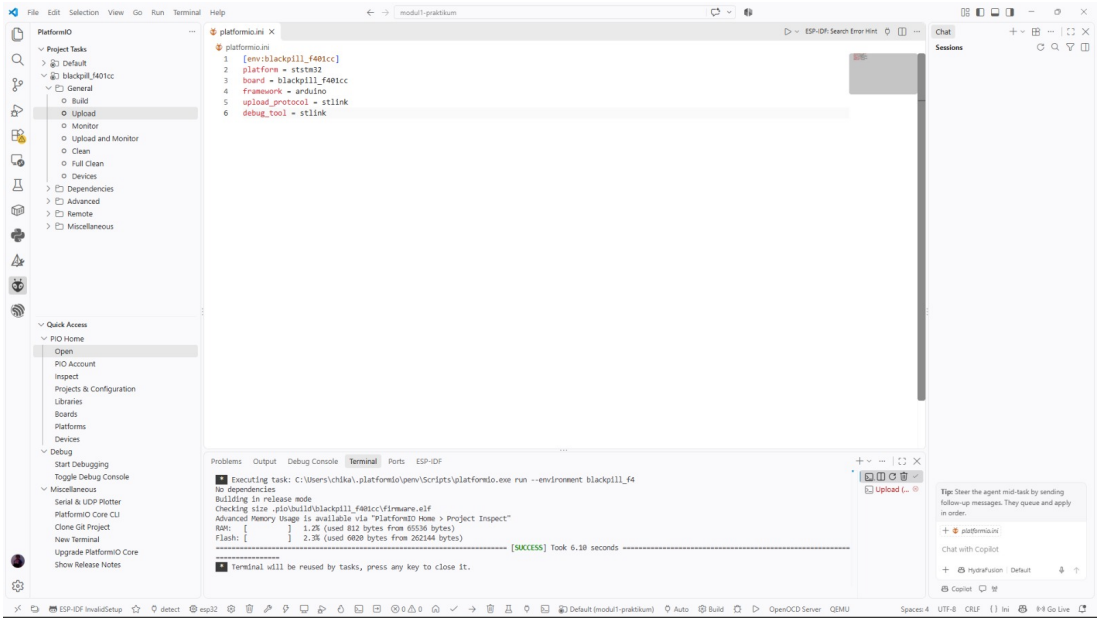
1.1 Persiapan Alat dan Bahan

Sebelum praktikum dimulai, perlu menyiapkan beberapa alat dan bahan yang akan digunakan, yaitu board STM32 Blackpill, board ESP32 DevKit, ST-Link V2, kabel USB, kabel jumper, breadboard, LED, pushbutton, resistor 10kΩ, dan sensor LDR. Laptop juga perlu mensetup Visual Studio Code dan ekstensi PlatformIO.

Seluruh percobaan dilakukan menggunakan satu project PlatformIO. Konfigurasi board dan framework disesuaikan melalui file `platformio.ini` sesuai dengan percobaan yang dilakukan.

1.2 Percobaan 1: Pengenalan STM32 Blackpill

1. Membuka project praktikum pada Visual Studio Code dan mengatur konfigurasi PlatformIO untuk board STM32 Blackpill dengan framework Arduino.

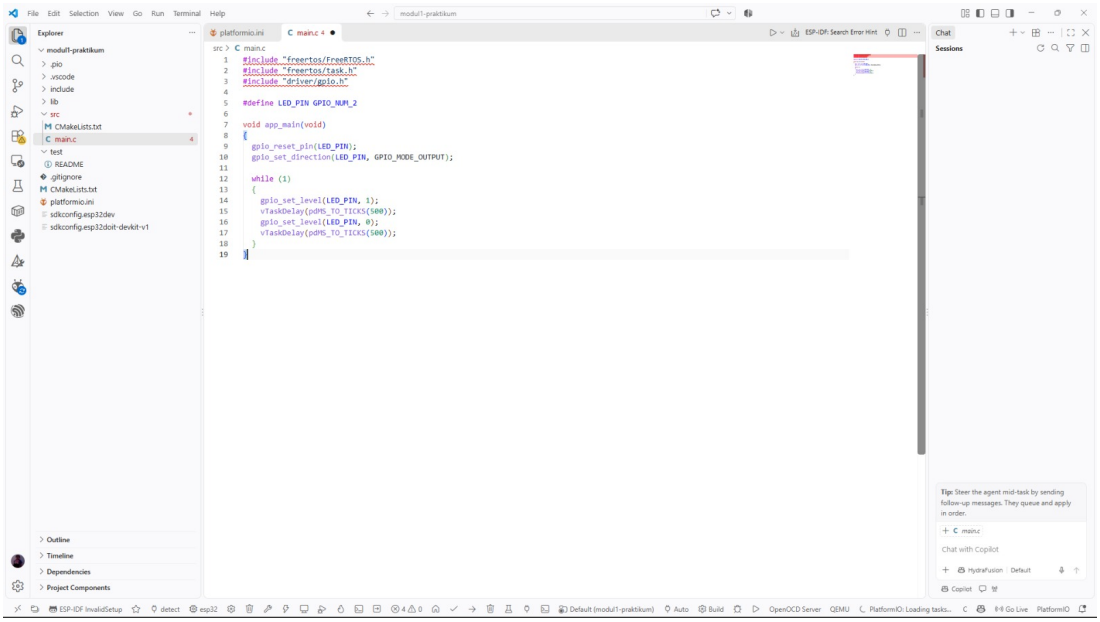


Gambar 1: Menyiapkan Project dan konfigurasi PlatformIO

2. Menghubungkan ST-Link ke board STM32 Blackpill menggunakan pin SWDIO, SWCLK, GND, dan 3.3V sesuai dengan koneksi yang tersedia.

3. Menghubungkan ST-Link ke laptop melalui kabel USB.

4. Menuliskan program Blink pada file `src/main.cpp` dengan menggunakan pin PC13 sebagai output LED onboard.

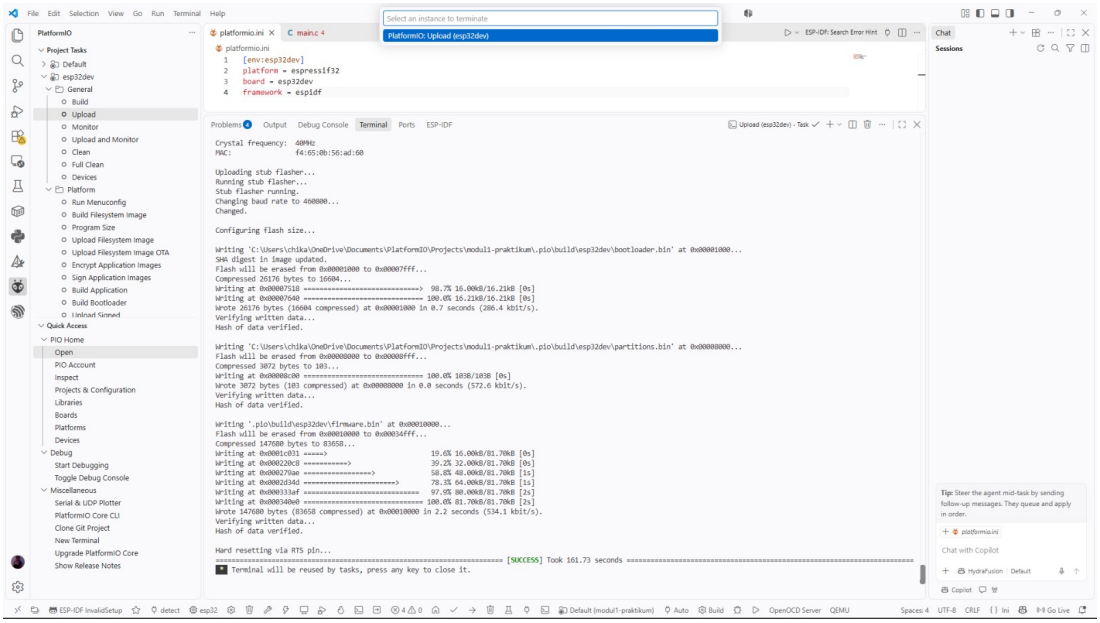


Gambar 2: Menulis Program Blink pada VScode dilanjut dengan build dan upload

5. Melakukan proses Build untuk memeriksa program, kemudian Upload untuk mengunggah program ke board menggunakan ST-Link.
6. Mengamati LED onboard pada STM32 Blackpill. Program diatur agar LED menyala selama 500 ms dan mati selama 500 ms secara berulang.

1.3 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF

1. Mengubah konfigurasi `platformio.ini` untuk menggunakan board ESP32 DevKit dengan framework ESP-IDF.



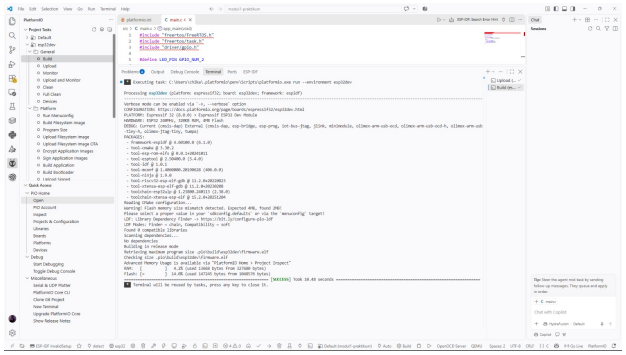
Gambar 3: Sukses mengupload ESP32 pada PlatformIO

2. Menghubungkan board ESP32 ke laptop menggunakan kabel USB dan memastikan board terdeteksi oleh komputer.

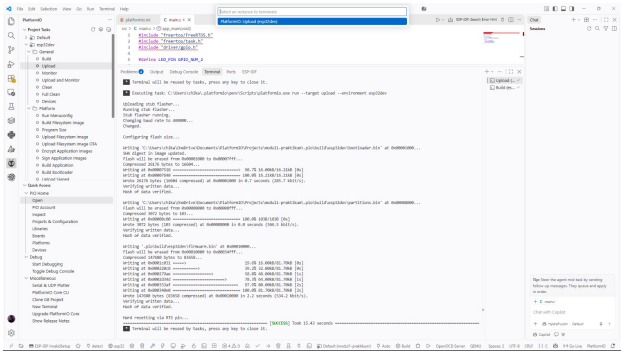
3. Menghapus file `src/main.cpp`, kemudian membuat file baru bernama `src/main.c`.

4. Menuliskan program Blink menggunakan fungsi GPIO dari ESP-IDF dengan pin GPIO 2 sebagai output LED.

5. Melakukan Build dan Upload untuk mengunggah program ke board ESP32.



Gambar 4: Berhasil build ESP-IDF



Gambar 5: Berhasil upload ESP-IDF

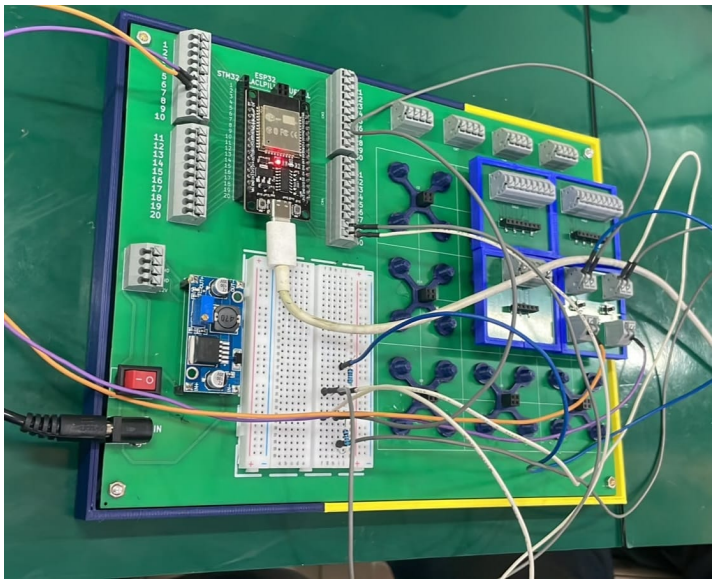
6. Mengamati LED onboard ESP32 untuk memastikan program berjalan dengan pola menyala dan mati setiap 500 ms.

1.4 Percobaan 3: Pull-up dan Pull-down

A. Konfigurasi Eksternal

1. Mengubah konfigurasi project menggunakan board ESP32 DevKit dengan framework Arduino.

2. Memasang dua pushbutton pada breadboard. Pushbutton pertama dihubungkan ke GPIO 32 menggunakan resistor pull-up 10kΩ, sedangkan pushbutton kedua dihubungkan ke GPIO 33 menggunakan resistor pull-down 10kΩ.



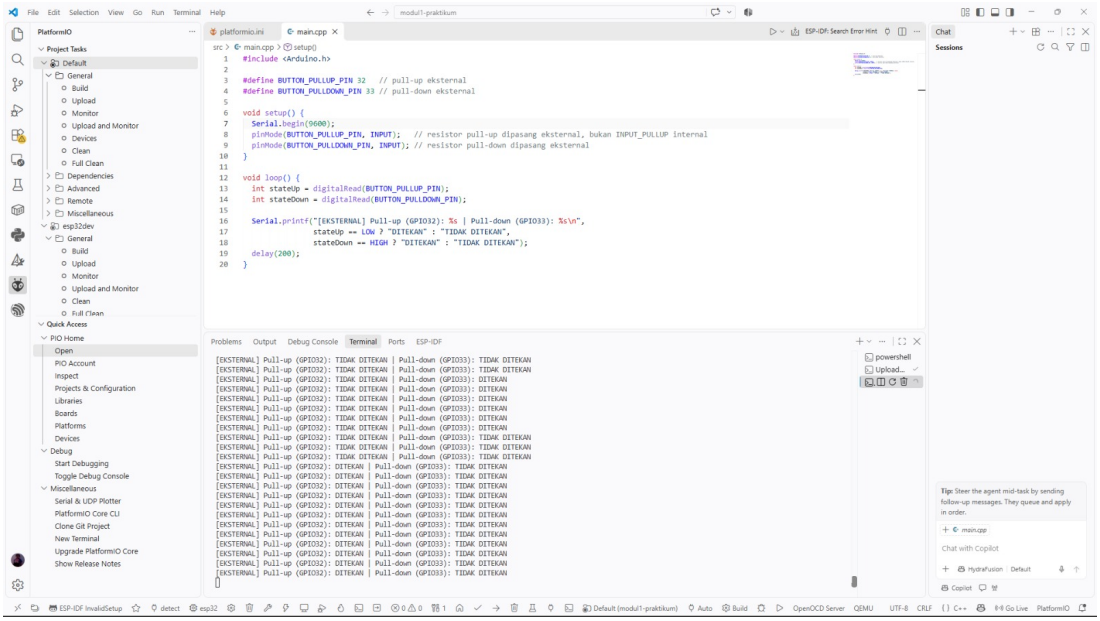
Gambar 6: Merangkai rangkaian sesuai modul

3. Menghubungkan rangkaian sesuai dengan skema pada modul. Rangkaian pull-up dihubungkan ke 3.3V melalui resistor dan tombol ke GND, sedangkan pull-down dihubungkan ke GND melalui resistor dan tombol ke 3.3V.

4. Menuliskan program untuk membaca kondisi kedua tombol menggunakan fungsi `digitalRead()`.

5. Melakukan Build dan Upload, kemudian membuka Serial Monitor dengan baud rate 115200.

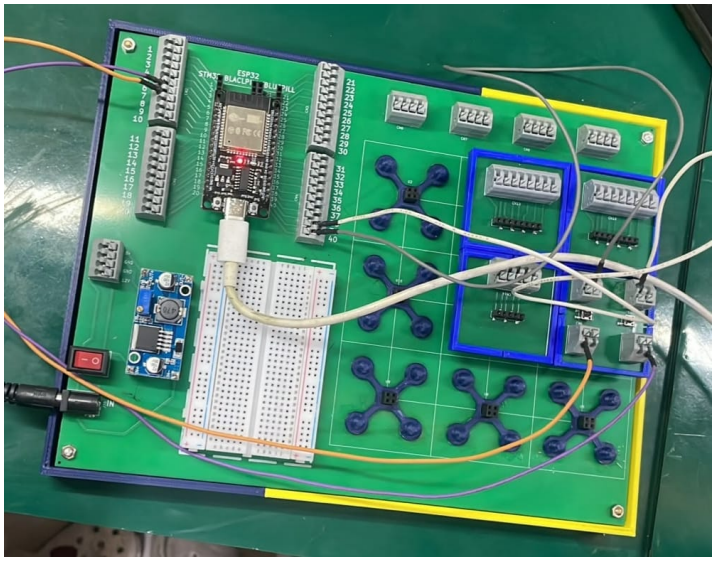
6. Menekan dan melepas kedua tombol secara bergantian untuk mengamati perubahan kondisi logika yang ditampilkan pada Serial Monitor.



Gambar 7: Hasil tombol dalam serial monitor

B. Konfigurasi Internal

1. Melepas resistor eksternal pada kedua rangkaian pushbutton, sementara posisi tombol dan koneksi GPIO tetap dipertahankan.



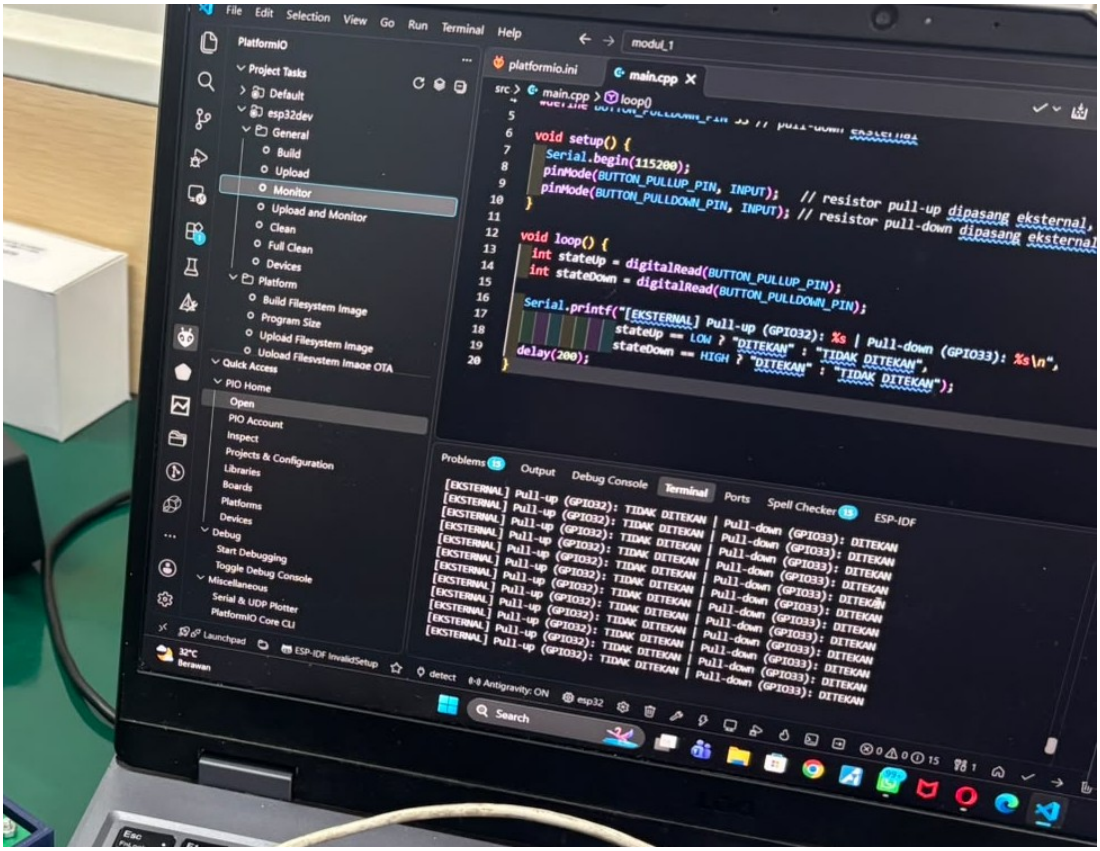
Gambar 8: Merangkai rangkaian sesuai modul

2. Mengubah konfigurasi pin menjadi `INPUT_PULLUP` pada GPIO 32 dan `INPUT_PULLDOWN` pada GPIO 33.

3. Melakukan Build dan Upload kembali ke ESP32.

4. Membuka Serial Monitor dan menguji kedua tombol dengan cara menekan serta melepasnya secara bergantian.

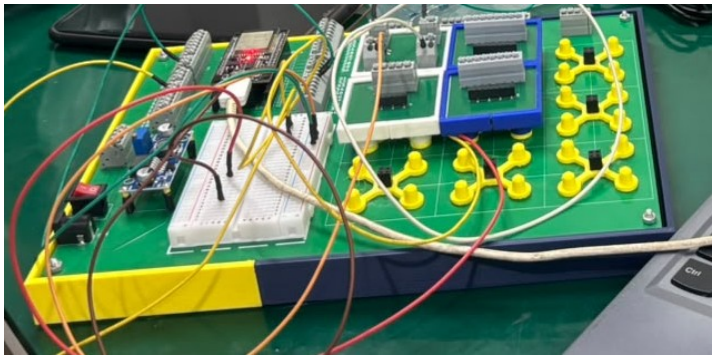
5. Membandingkan hasil pembacaan konfigurasi internal dengan hasil percobaan konfigurasi eksternal.



Gambar 9: Hasil tombol dalam serial monitor

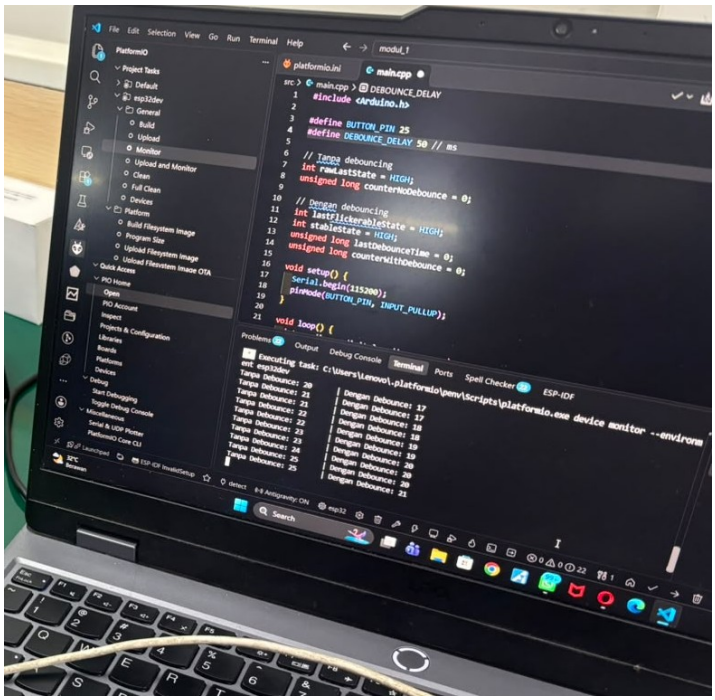
1.5 Percobaan 4: Debouncing pada Input Tombol GPIO

1. Memasang satu pushbutton pada GPIO 25 ESP32 dengan menggunakan konfigurasi pull-up internal.



Gambar 10: Merangkai rangkaian sesuai modul

2. Menuliskan program yang membandingkan dua metode pembacaan tombol, yaitu tanpa debouncing dan dengan debouncing.
3. Melakukan Build dan Upload, kemudian membuka Serial Monitor untuk mengamati nilai kedua penghitung.
4. Menekan tombol satu kali dan membandingkan perubahan nilai `counterNoDebounce` dengan `counterWithDebounce`.
5. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian melakukan Build dan Upload kembali.
6. Mengulangi pengujian dengan mengubah nilai `DEBOUNCE_DELAY` menjadi 200 ms.
7. Membandingkan hasil pengujian dengan waktu debouncing 10 ms, 50 ms, dan 200 ms untuk mengetahui pengaruh interval waktu terhadap pembacaan tombol.



Gambar 11: Menbandingkan hasil dengan dan tanpa Bounce

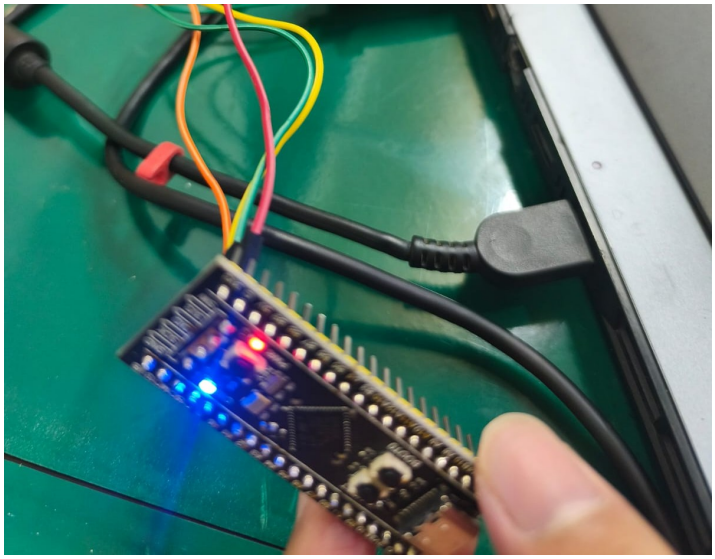
1.6 Percobaan 5: Pembacaan ADC dengan Sensor LDR

1. Menyusun rangkaian sensor LDR bersama resistor 10kΩ pada breadboard sebagai pembagi tegangan.
2. Menghubungkan salah satu kaki LDR ke sumber tegangan 3.3V. Kaki lainnya dihubungkan ke GPIO 34 dan resistor 10kΩ yang terhubung ke GND.
3. Mengubah konfigurasi project menggunakan board ESP32 DevKit dengan framework Arduino.
4. Menuliskan program untuk membaca nilai ADC menggunakan fungsi `analogRead()` pada GPIO 34.

5. Menambahkan perhitungan untuk mengubah nilai ADC menjadi perkiraan tegangan menggunakan rentang pembacaan 0–4095.

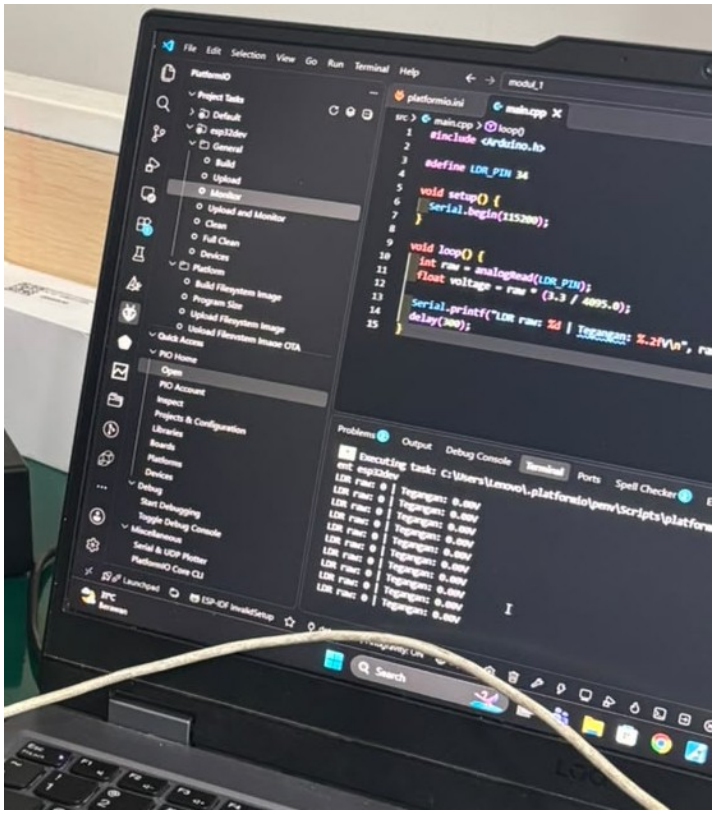
6. Melakukan Build dan Upload, kemudian membuka Serial Monitor dengan baud rate 115200.

7. Mengamati perubahan nilai ADC dan tegangan dengan menutup permukaan LDR menggunakan tangan, kemudian memberikan cahaya menggunakan lampu.



Gambar 12: Perubahan ADC dengan cahaya lampu

8. Membandingkan perubahan nilai pembacaan ADC berdasarkan perbedaan intensitas cahaya yang diterima LDR.



Gambar 13: Hasil nilai ADC pada serial monitor

2 Analisis Hasil Percobaan

Pada percobaan pertama dan kedua, dilakukan pengujian LED Blink menggunakan STM32 Blackpill dengan framework Arduino (pin PC13) dan ESP32 DevKit dengan framework ESP-IDF (GPIO2). Kedua program mengatur pin sebagai output sehingga LED menyala dan mati bergantian dengan jeda 500 ms. Pada STM32, LED bawaan umumnya aktif LOW sehingga menyala saat pin LOW, sedangkan pada ESP32 pengaturan waktu dilakukan menggunakan `vTaskDelay()` dari FreeRTOS. Hasil ini menunjukkan bahwa prinsip dasar pengendalian GPIO output sama pada kedua board, sedangkan perbedaannya terletak pada framework dan fungsi yang digunakan. Pada percobaan pertama sempat terjadi kendala karena PlatformIO bermasalah pada laptop awal sehingga proses upload tidak berjalan, dan masalah ini diatasi dengan menggunakan laptop lain.

Pada percobaan ketiga dan keempat, dilakukan pengujian input tombol pada ESP32. Pada percobaan ketiga, konfigurasi pull-up (GPIO32) dan pull-down (GPIO33) diuji menggunakan resistor eksternal maupun resistor internal (`INPUT_PULLUP` dan `INPUT_PULLDOWN`). Hasil pada Serial Monitor sesuai teori, yaitu pull-up menjaga kondisi awal pin HIGH dan pull-down menjaga kondisi awal pin LOW. Resistor internal membuat rangkaian lebih sederhana, sedangkan resistor eksternal memberi keleluasaan dalam menentukan konfigurasi. Pada percobaan keempat, pembacaan tombol di GPIO25 dibandingkan antara kondisi tanpa dan dengan debouncing menggunakan waktu 10 ms, 50 ms, dan 200 ms. Jumlah pembacaan dengan debouncing lebih sedikit karena sinyal tidak stabil akibat *bouncing* berhasil disaring, sehingga satu kali penekanan tidak terbaca berulang kali. Waktu debouncing yang terlalu kecil belum tentu menyaring bouncing dengan baik, sedangkan yang terlalu besar membuat respons tombol lebih lambat.

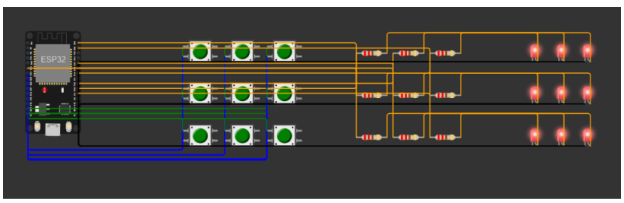
Pada percobaan kelima, sensor LDR dihubungkan ke GPIO34 dan dibaca menggunakan `analogRead()`, kemudian nilainya dikonversi menjadi tegangan. Nilai ADC yang terbaca berubah pada rentang 0 hingga 4095 dengan tegangan sekitar 0 hingga 3,3 V, sesuai dengan resolusi ADC 12-bit pada ESP32. Perubahan ini terjadi karena hambatan LDR berubah mengikuti intensitas cahaya sehingga tegangan pada rangkaian pembagi tegangan ikut berubah, dengan arah perubahan bergantung pada susunan LDR dan resistor. Percobaan ini menunjukkan bahwa mikrokontroler dapat membaca sinyal analog dari sensor untuk mengetahui kondisi lingkungan. Selama praktikum, kendala pada PlatformIO di laptop awal juga menunjukkan bahwa kesiapan perangkat dan lingkungan pengembangan memengaruhi kelancaran praktikum.

3 Hasil Tugas Modul

Pembuatan Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3 dengan simulasi WOKWI :

```
Matrix Pushbutton 3x3 -> Matrix LED 3x3
Tombol (1,1) -> LED ON
Tombol (1,1) -> LED OFF
Tombol (1,2) -> LED ON
Tombol (1,2) -> LED OFF
Tombol (1,3) -> LED ON
Tombol (1,3) -> LED OFF
Tombol (2,1) -> LED ON
Tombol (2,1) -> LED OFF
Tombol (2,2) -> LED ON
Tombol (2,2) -> LED OFF
Tombol (2,3) -> LED ON
Tombol (2,3) -> LED OFF
Tombol (3,1) -> LED ON
Tombol (3,1) -> LED OFF
Tombol (3,2) -> LED ON
Tombol (3,2) -> LED OFF
Tombol (3,3) -> LED ON
Tombol (3,3) -> LED OFF
```

Gambar 14: Hasil simulasi pada WOKWI



Gambar 15: Rangkaian Pada WOKWI

Berikut kode yang digunakan :

```
#include <Arduino.h>

const int rowPins[3] = {13, 14, 27};
const int colPins[3] = {26, 25, 33};
const int ledPins[3][3] = {
  {16, 17, 18},
  {19, 21, 22},
  {23, 32, 4}
};

const unsigned long debounceDelay = 50;

bool lastReading[3][3];
bool stableState[3][3];
unsigned long lastChange[3][3];
bool ledOn[3][3];

void scanMatrix(bool raw[3][3]) {
  for (int r = 0; r < 3; r++) {
    for (int i = 0; i < 3; i++) digitalWrite(rowPins[i], HIGH);
    digitalWrite(rowPins[r], LOW);
    delayMicroseconds(10);
    for (int c = 0; c < 3; c++) {
      raw[r][c] = (digitalRead(colPins[c]) == LOW);
    }
  }
  for (int i = 0; i < 3; i++) digitalWrite(rowPins[i], HIGH);
}

void setup() {
  Serial.begin(115200);

  for (int i = 0; i < 3; i++) {
    pinMode(rowPins[i], OUTPUT);
    digitalWrite(rowPins[i], HIGH);
    pinMode(colPins[i], INPUT_PULLUP);
  }

  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      pinMode(ledPins[r][c], OUTPUT);
      digitalWrite(ledPins[r][c], LOW);
      lastReading[r][c] = false;
      stableState[r][c] = false;
      lastChange[r][c] = 0;
      ledOn[r][c] = false;
    }
  }

  Serial.println("Matrix Pushbutton 3x3 -> Matrix LED 3x3");
}

void loop() {
```

```
54      bool raw[3][3];
55      scanMatrix(raw);
56      unsigned long now = millis();
57
58      for (int r = 0; r < 3; r++) {
59        for (int c = 0; c < 3; c++) {
60          if (raw[r][c] != lastReading[r][c]) {
61            lastChange[r][c] = now;
62            lastReading[r][c] = raw[r][c];
63          }
64
65          if ((now - lastChange[r][c]) > debounceDelay && raw[r][c] != stableState[r][c
    ]) {
66            stableState[r][c] = raw[r][c];
67
68            if (stableState[r][c]) {
69              ledOn[r][c] = !ledOn[r][c];
70              digitalWrite(ledPins[r][c], ledOn[r][c] ? HIGH : LOW);
71              Serial.printf("Tombol (%d,%d) -> LED %s\n", r + 1, c + 1, ledOn[r][c] ? "
    ON" : "OFF");
72            }
73          }
74        }
75      }
76    }
77
```

Jawaban Pertanyaan Analisis :

1. Bouncing terjadi di dalam tombolnya sendiri, bukan karena banyaknya tombol yang ditekan. Seperti pada teori C.5, kontak logam tombol mekanis memantul beberapa kali dalam waktu sangat singkat saat ditekan atau dilepas, sehingga sinyalnya bolak-balik HIGH dan LOW sebelum stabil. Karena ESP32 membaca jauh lebih cepat dari durasi bouncing, satu kali tekan bisa terbaca beberapa kali. Pada program ini LED berubah (toggle) tiap penekanan, jadi tanpa debouncing LED bisa berganti ON/OFF berkali-kali dan hasil akhirnya salah. Dengan debouncing, LED baru berubah setelah pembacaan stabil 50 ms.

2. Kolom memakai pull-up internal (INPUT_PULLUP), sehingga saat tombol tidak ditekan pin bernilai HIGH, dan saat ditekan pin terhubung ke baris yang LOW sehingga bernilai LOW. Pin tidak boleh dibiarkan floating karena mudah terpengaruh noise, sehingga nilainya bisa berubah acak antara HIGH dan LOW dan LED bisa berubah sendiri tanpa ada tombol ditekan.

3. Digital input hanya membandingkan tegangan dengan batas ambang sehingga hanya ada dua kondisi, HIGH atau LOW (1 bit). ADC 12-bit pada ESP32 mengubah tegangan 0–3,3 V menjadi 4096 tingkat nilai (0–4095). Jika tombol diganti LDR, tegangan berubah kontinu sesuai intensitas cahaya, sehingga ADC dapat membedakan banyak tingkat terang-gelap, bukan hanya terang atau gelap.

4 Kesimpulan

Praktikum ini berhasil dilakukan dan seluruh hasilnya sesuai dengan teori. Praktikan dapat memprogram STM32 Blackpill dan ESP32 menggunakan PlatformIO, mengatur GPIO sebagai output maupun input, serta memahami fungsi pull-up dan pull-down, debouncing, dan pembacaan ADC pada sensor LDR. Pada tugas modul, matrix pushbutton 3x3 berhasil mengendalikan matrix LED 3x3 pada simulasi dengan debouncing sehingga LED hanya berubah setelah pembacaan tombol stabil. Kendala PlatformIO pada laptop awal dapat diatasi dengan berpindah ke laptop lain.

5 Lampiran

5.1 Dokumentasi saat praktikum

